



Ranking Candidates the Way a Great Recruiter Would

The Data & AI Challenge — Intelligent Candidate Discovery & Ranking

Team Name : FILL_ME

Team Leader : FILL_ME

Problem Statement : Rank 100,000 candidate profiles against the Senior AI Engineer JD; deliver a trusted, explainable top-100 shortlist.

Solution Overview

Evidence-graded recruiter model: $\text{final score} = \text{Content} \times \text{Fit} \times \text{Availability} + \text{Prior}$

- Content — what the candidate has actually built, read from career-history text against a graded evidence rubric (production retrieval / ranking / recsys highest).
- Fit — the JD's explicit rules: 5-9 yr band, Pune/Noida-first location ladder, consulting-only and job-hopper demotions.
- Availability — response rate, activity recency, notice period: a perfect profile that ignores recruiters is not a real candidate (the JD says exactly this).
- Prior — recruiter-demand signals: search appearances, recruiter saves, verified retrieval-stack assessments, public GitHub activity.

What differentiates it

- Skills lists and self-summaries earn ZERO positive score — they are where fake profiles concentrate keywords. Career history is the only field that can't be cheaply faked.
- Five profile-consistency checks exclude impossible profiles (the dataset's honeypots) before scoring; every ranking decision is rule-traceable and auditable.

JD Understanding & Candidate Evaluation

What the JD actually asks for (beyond the keyword list)

- Must-haves: production embeddings/retrieval experience, vector-search operations, ranking evaluation (NDCG/MRR, offline-to-online), strong Python — shipped to real users.
- Explicit anti-patterns: research-only careers, <12-month LangChain-wrapper experience, consulting-firm-only careers, CV-only without NLP/IR, title-chasing job-hoppers.
- Logistics matter: Pune/Noida preferred (Hyd/Mum/NCR welcome), sub-30-day notice loved, behavioral availability explicitly called out.

Signals ranked by importance in our evaluation

- 1. Career-history evidence of retrieval/ranking systems in production (decisive)
- 2. Behavioral availability: recruiter response rate, last-active recency, notice period
- 3. JD fit: experience band, location/relocation, employer-type history
- 4. Corroboration: skill durations & endorsements, assessment scores, GitHub activity
- Keyword presence in skills/headline: weight ZERO — by design.

Ranking Methodology

`candidates.jsonl` → features (55) → hard exclusions → $C \times F \times A + P$ → top-100 + reasoning

- Retrieve: single streaming pass extracts 55 features per candidate — career evidence, consistency markers, skill-trust metrics, 23 behavioral signals, logistics.
- Score: graded evidence rubric over career-history text (graded against the JD, then blind re-graded by a 3-judge panel; disagreements reconciled by median).
- Skill claims count only when corroborated: long durations, real endorsements, matching assessment scores. Uncorroborated keyword lists contribute nothing.
- Combine: content score scaled by two multipliers (fit, availability) plus a small additive engagement prior. Multiplicative availability chosen via simulation — additive weighting lets behaviorally-dead profiles crowd out hireable ones.
- Heuristics over models at rank time: deterministic rules + pandas/regex. No model inference, no embeddings, no network. 100k candidates in ~15 seconds on CPU.

Explainability & Data Validation

Explainability

- Every candidate's reasoning is generated from their own profile facts: strongest career evidence, concrete signal values (response %, notice days), and honest concerns.
- No claim can appear that is not in the profile — the generator only composes from extracted fields, so unsupported justifications are structurally impossible.
- Tone follows rank: top picks read strong; rank-90s say honestly why they are borderline.

Suspicious-profile handling (the planted traps)

- Honeypots: 5 independent impossibility checks (expert skills with 0 months; durations contradicting dates; experience > career span; summary vs field mismatch; future certs).
- Keyword stuffers: non-technical titles carrying 7–13 AI skills with zero ML evidence in career history — excluded; cloned skill-lists deduplicated to the coherent member.
- Ghosts: elite text + dead signals (response <20%, inactive >100d) — hard-demoted.
- Final assembly re-asserts: zero honeypot markers in the top-100, zero stuffers, monotone scores, validator-compliant tie-breaks.

End-to-End Workflow

```
JD analysis (one-time, human + AI-assisted)
└─ graded evidence rubric + JD fit rules → config/params.json

python rank.py --candidates candidates.jsonl --out submission.csv
1. stream-parse 100k profiles          (~10 s)
2. derive 55 features per candidate    (memoized text analysis)
3. hard exclusions                     (~31k profiles removed)
4. score C x F x A + P, sort, tie-break
5. generate per-candidate reasoning
6. assembly assertions + write CSV     (~15 s total)
```

- One command, deterministic output, no pre-computation step required.
- Sandbox (Streamlit) runs the identical pipeline on any uploaded sample.

System Architecture

```
rank.py (CLI)
├── src/features.py    streaming JSONL -> 55-feature table
│   ├── title taxonomy, evidence lexicons, consistency markers
│   └── memoized text scans (templated corpus -> ~1000x fewer regex calls)
├── src/scoring.py     exclusions -> C x F x A + P -> ranked 100
│   └── all weights/grades externalized to config/params.json
├── src/reasoning.py   profile facts -> recruiter-style explanation
└── config/params.json the full rubric: 44 graded evidence patterns,
                        fit ladders, availability steps, prior weights
```

- Pure pandas + numpy + regex. No service dependencies, no GPU, no network, < 4 GB RAM.
- Same code path powers the CLI and the hosted sandbox.

Results & Performance

Ranking quality

- Top-100 is 100% candidates with production-ML evidence in career history (99% India, all within the JD's experience band after consistency-checked years-of-experience).
- Catches all 8 'plain-language' senior ranking engineers (zero buzzwords in their profiles) in the head of the list — the trap naive keyword/embedding rankers miss.
- Zero honeypots in the top-100 (verified by independent re-derivation from raw data in an adversarial audit); keyword stuffers score zero by construction.
- Reasoning passes all six Stage-4 checks by construction: specific, JD-connected, honest about gaps, hallucination-free, varied, rank-consistent.

Compute (constraint: 5 min / 16 GB / CPU-only / no network)

- ~15 seconds end-to-end for 100,000 candidates on a laptop CPU — 20x headroom.
- < 4 GB peak memory, zero network calls, deterministic across runs.

Technologies Used

- Python 3.13, pandas, numpy — the entire ranking pipeline. Chosen for determinism, auditability, and CPU speed: the task is evidence reading, not similarity search.
- Regex evidence lexicons + graded rubric (config/params.json) — transparent, reviewable scoring; every rank is explainable line-by-line.
- Streamlit — hosted sandbox running the identical pipeline on uploaded samples.
- Claude (AI-assisted engineering, declared) — EDA at scale, blind multi-judge rubric grading, adversarial QA. All outputs verified by re-derivation from raw data.

Why NOT embeddings / LLM ranking at inference

- Recruiting corpora are adversarial: keyword-stuffed profiles and near-duplicate texts reward surface similarity. Embedding rankers score the traps highly, miss plain-language evidence, and cost ~1000x more compute. Reading structured evidence wins on quality, speed, explainability — and scales to 200k+ candidates in production.

Submission Assets

- GitHub repository: https://github.com/FILL_ME/redrob-ranker
- Hosted sandbox (Streamlit): https://FILL_ME.streamlit.app
- Ranked output: `submission.csv` (top-100, validator-clean)
- Reproduce: `python rank.py --candidates ./candidates.jsonl --out ./submission.csv`
- Methodology & analysis: `README.md` + `eda/` (two analysis rounds, 3-judge rubric calibration, adversarial audit of the final list — full provenance for every parameter)

Built to be defended: every scoring rule traces to a sentence in the JD or a measured property of the data.